

ELTE IK - Computer Science BSc

Final Exam Topics

16. Advanced algorithms

Note: This document was translated from Hungarian to English by AI. Please verify the information against the original Hungarian source before relying on it.

Advanced algorithms

Elementary graph algorithms: breadth-first, depth-first traversal and its applications. Minimum spanning trees, a general algorithm, Kruskal's and Prim's algorithms. Single-source shortest paths, queue-based Bellman-Ford, Dijkstra, DAG shortest path. All-pairs shortest paths: Floyd-Warshall algorithm. Transitive closure of a graph.

1 Graph algorithms

1.1 Graph representation

Linked-list representation

Let us place the vertices of the graph in an array (or linked list). To every element let us assign a linked list in which we list the neighbors of the given vertex (with the possible edge weights).

Matrix representation

Let the number of vertices be n . Then in a matrix $A^{n \times n}$ we mark which vertices are connected. Then both in the rows and in the columns the vertices appear, and in cell a_{ij} the weight of the edge leading from vertex i to vertex j appears; if there is no edge between the two vertices, then $-\infty$ (in the unweighted case 1 and 0)

If the graph is undirected, obviously $a_{ij} = a_{ji}$

1.2 Breadth-first traversal

We reach the vertices in the order of distance from the start vertex s of a (directed/undirected) graph G . It gives the spanning tree of the shortest paths, so only the distance matters, not the weight.

We store the tracked vertices in a queue data structure; we put the children of the current vertex into the queue. The next vertex will be the very first element of the queue.

We write the distances of the vertices into a d array, their parents into a π array, and initialize with ∞ and 0 respectively.

The algorithm:

1. We put the start vertex s into the queue
2. We repeat the following steps until the queue is empty
3. We take out the very first (u) element of the queue
4. Those child vertices whose distance is not ∞ we disregard (we have already been on these)

5. For the other children (v): we set their parent ($\pi[v] = u$), and their distance ($d[v] = d[u] + 1$). Then we put them into the queue.

Its applications

1. Finding the shortest path between two nodes u and v , measuring the length of the path by the number of edges (advantage compared to depth-first search)
2. (Reverse) Cuthill–McKee mesh numbering, a variant of breadth-first traversal
3. Serialization/deserialization of a binary tree versus serialization in sorted order; it enables the efficient reconstruction of the tree
4. Testing the bipartiteness of a bipartite graph

1.3 Searching for minimum-cost paths

Dijkstra algorithm

In a directed or undirected graph G with positive edge weights, it searches for minimum-cost paths from the start vertex s to every vertex.

The algorithm is a modified version of breadth-first traversal. Since here a longer path can have a smaller cost than a shorter one, an already-found vertex must not be disregarded. Therefore every vertex has three states (not reached, reached, done). We handle the d and π arrays similarly to breadth-first traversal.

The not-yet-done vertices we place into a priority queue, or in every case we apply minimum search.

The algorithm:

1. We set the weight of the start vertex s to 0 and store it
2. We repeat the following steps until our container is empty
3. We take out the best (u) element of the queue, and set it to “done”
4. If a child vertex (v) is not done, and the weight of the path currently leading to it is smaller than the previous one, then: we set its parent to u ($\pi[v] = u$), and update its weight ($d[v] = d[u] + d(u, v)$).
5. The other vertices we skip.

Bellman-Ford algorithm

From the start vertex s of an edge-weighted (even negative) directed graph G it searches for minimum-cost paths to every edge, and recognizes if there is a negative-cost cycle in the graph. We handle the d and π arrays similarly to the previous ones.

The algorithm:

1. Let us set the weight of the start vertex to 0.
2. In $n - 1$ iterations let us go through all vertices, and compare every vertex (u) with every vertex (v).
If we found a cheaper path then in v we overwrite its weight ($d[v] = d[u] + d(u, v)$), and its parent ($\pi[v] = u$).
3. If in the n -th iteration there was also a modification, there is a negative cycle in the graph

1.4 Searching for a minimum-cost spanning tree

The minimum-cost spanning tree or minimum spanning tree is the spanning tree of smallest edge weight in a connected, undirected graph. A spanning tree is a tree that contains all vertices of the graph and whose edges are among the edges of the original graph. The minimum spanning tree is not necessarily unique, but its weight is. For searching for any minimum spanning tree of a graph, Kruskal’s and Prim’s algorithms can be used. Both algorithms use a greedy strategy.

Kruskal algorithm

We sort the edges in increasing order of their weight, and examine in turn which ones we may include in the solution. Initially every vertex of the graph forms a separate set. An examined edge gets included in the solution if its two endpoints are in two different sets, because from this we know that adding the examined edge does not create a cycle in the graph. Then we merge the two sets. At the end of the algorithm a single set will remain.

The algorithm:

1. Let us choose the edge of smallest weight.
2. If adding the edge to the subgraph forms a cycle, let us discard it, otherwise let us add it.
3. Let us repeat the above steps until there is an unexamined edge.

Its applications:

1. The Kruskal algorithm was made for the simple proof of the theorem that the minimum-cost spanning tree is unique if there are no two edges of equal weight in the graph.
2. A random maze can be generated with it. In this case the weight of every edge of the graph to be processed coincides, so its edges do not have to be sorted.

Prim algorithm

The Prim algorithm searches for a minimum-cost spanning tree from the start vertex s of an undirected edge-weighted (even negative) graph. We handle the d and π arrays similarly to the previous ones. The algorithm places the vertices into a priority queue.

The algorithm:

1. Let us set the weight of the start vertex to 0.
2. We place the vertices into the priority queue.
3. We perform the following steps until the priority queue is emptied.
4. We take out a vertex (u) from the queue.
5. For every child of it (v) that is still in the queue and the weight of the recorded edge leading to v is greater than the one just found: We change the parent of v to u , overwrite the recorded weight $d[v] = d(u, v)$. Then we overwrite the state of v in the priority queue.
6. With those children that are not in the queue, or whose weight we cannot improve, we do not change anything.

ALTERNATIVELY

Its operating principle is that it builds the tree vertex by vertex, starting from an arbitrary vertex and at every step finding the cheapest possible edge to a next vertex.

The algorithm:

1. Let us choose an arbitrary vertex of the graph; let this be a single-vertex tree.
2. As long as there is a vertex of the original graph that is not in the tree, let us repeat the following steps.
3. Let us choose the one of smallest weight among the edges running between the vertices of the tree and the other vertices of the graph.
4. Let us move the non-tree vertex of the chosen edge into the tree together with the edge.

Application: This too can be used for generating a random maze.

1.5 Depth-first traversal

By the depth-first traversal of a directed (not necessarily connected) graph G we obtain a depth-first tree (forest). The algorithm is the following:

- The edge weights play no role
- There is no start vertex; we start the algorithm for every vertex of the graph. (Of course, then, if we choose a vertex we have already been on, the algorithm does not start.)
- We choose the vertices greedily, that is, choosing the first among the children of every vertex, we proceed forward as long as possible. (We find a vertex that has no child, or whose every child we have already been on.)
- If we can no longer proceed forward, we step back.
- To every vertex we assign two values. One is the depth-first number, which marks how many-th we reached it. The other is the finishing number, which marks how many-th we stepped back from it.

The edges of the graph can be classified during the depth-first traversal. (At initialization we set every value to 0)

- Tree edge: The depth-first number of the next vertex is 0
- Back edge: The depth-first number of the next vertex is greater than 0, and its finishing number is 0 (So we turn back to a previous vertex of the current path)
- Cross edge: The depth-first number of the next vertex is greater than 0, and its finishing number is also greater than 0, and furthermore the depth-first number of the current vertex is greater than the depth-first number of the next vertex. (Then we are dealing with an edge starting from a vertex preceding the current vertex and pointing into an already-found path)
- Forward edge: The depth-first number of the next vertex is greater than 0, and its finishing number is also greater than 0, and furthermore the depth-first number of the current vertex is smaller than the depth-first number of the next vertex. (Then we are dealing with an edge starting from the current vertex and pointing into an already-found path)

Algorithms that use depth-first traversal as a building block

1. Finding the connected component of a graph
2. Topological sorting
3. Finding 2- (edge or vertex) connected components
4. Finding 3- (edge or vertex) connected components
5. Finding separating edges in a graph
6. Finding the strongly connected components of a graph
7. Solving puzzles with only one solution, for example mazes
8. The creation of a maze can use randomly performed depth-first search

1.6 Topological sorting of a DAG

Basic concepts

- Topological sorting:
A topological sorting of a graph $G(V, E)$ is an order of the vertices in which for $\forall(u \rightarrow v) \in E$ edge u is earlier in the order than v
- DAG - Directed Acyclic Graph:
A directed acyclic graph.
Most often used for directing workflows and analyzing dependencies.
Properties:
 - If for a graph G the depth-first traversal finds a back edge (that is, found a cycle) $\implies G$ is not a DAG
 - If G is not a DAG (there is a cycle in it) $\implies$ Any depth-first traversal finds a back edge
 - If G has a topological sorting $\implies G$ is a DAG
 - Every DAG can be topologically sorted.

Topological sorting of a DAG

During the depth-first traversal of a graph G let us put into a stack those vertices from which we stepped back. After the algorithm, writing out the content of the stack, we obtain a topological sorting of the graph.

1.7 All-pairs shortest paths: Floyd-Warshall algorithm

We write the distances of the vertices into a d matrix, their parents into a π matrix, which we initialize, similarly to the previous ones, with ∞ and 0 values. The end result will be that $d[i, j]$ is the length of the optimal path leading from the vertex with index i to the vertex with index j , or ∞ if there is no path from i to j . And $\pi[i, j]$ is, computed by the algorithm, the direct predecessor of vertex j on the optimal path leading from the vertex with index i to the vertex with index j , if $i \neq j$ and a path from i to j exists, otherwise $\pi[i, j] = 0$. **Description of the algorithm:**

Take a graph G , with vertices V numbered from 1 to N . Furthermore a function, the so-called **shortestPath(i,j,k)**, which returns the shortest path between i and j using given vertices: $1, 2, \dots, k$ as intermediate points along the vertices. Taking the given function into account, our goal is to find the shortest path from every i to j using any of the vertices $1, 2, \dots, N$. For each of the vertex pairs, the **shortestPath(i,j,k)** can be either

(1) a path that does not pass through k (which uses only the given vertices: $1, \dots, k-1$)

OR

(2) a path that passes through k (from i to k and then from k to j , in both cases using the given intermediate vertices: $1, \dots, k-1$).

We know that the best path from i to j that uses only those vertices which are from 1 through $k-1$ is determined by the **shortestPath(i,j,k-1)**, and it is clear that if there were a better path from i to k and then from there to j , then the length of these paths would be the concatenation of the shortest path from i to k (using only the intermediate vertices $1, \dots, k-1$) and the shortest from k to j (using only the intermediate vertices $1, \dots, k-1$).

If $\omega(i, j)$ is the weight of the edge between the given vertices i and j , then we can determine the **shortestPath(i,j,k)** function according to the following recursive formula: **shortestPath(i,j,0) = $\omega(i, j)$** recursive case:

shortestPath(i,j,k) =

min(shortestPath(i,j,k-1), shortestPath(i,k,k-1) + shortestPath(k,j,k-1)).

This formula is the heart of the Floyd-Warshall algorithm. During the run of the algorithm it first computes, based on

shortestPath(i,j,k), the (i, j) pairs for the cases $k = 1$, then $k = 2$ and so on. This process continues until finally $k = N$ holds and the N -th iteration also runs, and we have found the shortest path for each (i, j) pair using any intermediate vertex.

1.8 Transitive closure of a graph

First definition: By the transitive closure of the graph $G = (V, E)$ we mean the relation $V \times V \supseteq T$, where $(u, v) \in T$ if in G the vertex v is reachable from the vertex u .

Second definition: The transitive closure of a directed graph $G = (V, E)$ is the graph $G' = (V, E')$ in which an edge leads from u to v exactly when a directed path leads from u to v in the original graph G .

To every directed acyclic graph (**DAG**) a partial order of its vertices can be assigned, in which $u \leq v$ holds exactly when in the graph there exists a directed path from u to v . At the same time, such a partial order can be represented by many different directed acyclic graphs. Of these, the transitive reduction contains the fewest edges, and the transitive closure the most.

The transitive closure of a graph can easily be computed, e.g., with the Floyd-Warshall algorithm in $O(n^3)$ time, or with n breadth-first searches in $O(n(n+m))$ time. At the same time there are also significantly more efficient methods that in practice run in almost linear time.

One such method is based on the determination of the strongly connected components, as well as on the topological sorting of the graph of the components, but it also applies complex data structures.